

## CSCE 678 DISTRIBUTED SYSTEMS AND CLOUD COMPUTING · MODULE 05 · LECTURE NOTES

# The Impossibility Results

*What they actually say, and what they are quoted as saying.*

## MODULE OUTCOMES

- State CAP precisely and say what it does not claim.
- State FLP precisely and explain why consensus still works.
- Explain PACELC and why it is the more useful framing.
- Explain the two generals and the coordinated attack problem.
- Recognise these results being misused.

## 1 CAP

### CAP DOES NOT SAY 'PICK TWO'

**The theorem's actual claim is narrow: a system that is experiencing a network partition cannot be both consistent and available.** That is the whole statement.

**'Consistent' means linearizable** (Module 04 §2) — not 'correct', not ACID, not any of the weaker models. A causally consistent system is not 'inconsistent' in CAP's sense of giving something up; it is simply not linearizable.

**'Available' means every non-failing node returns a response to every request.** This is far stronger than the operational sense of 'available' — a system with 99.99% uptime is not CAP-available if any node ever refuses a request.

**And partitions are not optional.** You do not get to choose whether your network partitions; the network chooses, and it will. **So the only choice is what to do during one**, and 'CA' is not a selectable category — a system claiming it is simply one that has not decided what it does when the partition arrives, which means it will do something arbitrary.

| What is said              | What is true                                                                                                                                                                                    |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'Pick two of the three.'  | <b>P is not a choice. You pick C or A, and only during a partition.</b> The 'pick two' formulation is the single most damaging piece of folklore in this subject.                               |
| 'NoSQL is AP; SQL is CA.' | <b>Both are configurable, and most real systems are neither purely.</b> Cassandra can be configured for quorum reads and writes; PostgreSQL with synchronous replication makes a CP-ish choice. |

| What is said                               | What is true                                                                                                                                                                        |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'We chose availability.'                   | Frequently means 'we did not think about it and the default was eventual consistency'.                                                                                              |
| 'CAP forbids strong consistency at scale.' | <b>Only during a partition.</b> Spanner is linearizable across continents and simply becomes unavailable in the minority partition, which is a deliberate and documented choice.    |
| 'So we must use eventual consistency.'     | <b>Causal consistency is strictly stronger and is also available under partition</b> (Module 04 §2). Eventual is not the only partition-tolerant option and is rarely the best one. |

Brewer published **'CAP Twelve Years Later: How the Rules Have Changed'** largely to correct the folklore version of his own conjecture, which is about as strong a signal as one can get that the popular reading is wrong.

## 2 FLP

### NO DETERMINISTIC CONSENSUS IN AN ASYNCHRONOUS SYSTEM

Fischer, Lynch and Paterson proved that in a fully asynchronous system with even one possible crash failure, no deterministic algorithm guarantees consensus.

'Asynchronous' is doing heavy lifting: it means no bound whatsoever on message delay or on processing speed. In that model a slow node is formally indistinguishable from a dead one — which is Module 01 §1's timeout ambiguity elevated into a theorem.

The proof constructs an infinite execution in which the algorithm never decides, by repeatedly delaying whichever message would resolve the ambiguity. It does not show the algorithm decides wrongly — it shows it may never decide at all.

So the impossibility is about *termination*, not safety. That distinction is the entire reason practical consensus exists, and it is the part the folklore version ('consensus is impossible') drops.

**WHY RAFT AND PAXOS WORK ANYWAY**

**They never violate safety.** Two conflicting values are never decided, under any pattern of delays, losses, and crashes whatsoever. That guarantee is unconditional and is what you actually need.

**They sacrifice guaranteed liveness.** Under sufficiently adversarial timing — an election that repeatedly splits, messages delayed exactly enough to trigger timeouts — they can fail to make progress indefinitely.

**Which is precisely what FLP says must be possible.**

**And they add randomness.** Raft's randomised election timeouts break the symmetry that the FLP adversary depends on, so progress occurs with probability 1 even though it is not guaranteed in the worst case. **Randomisation does not contradict FLP**, which is a statement about deterministic algorithms.

**And real networks are *partially synchronous*** — usually timely, occasionally not, with no bound you can state in advance but with long periods of good behaviour. **So the algorithms are designed to be safe always and live during the good periods**, which is exactly the right engineering response to an impossibility result: **find the assumption that makes it bite, and note that reality mostly does not satisfy it.**

### 3 PACELC

**if (P) then (A or C) else (L or C)**

**If there is a Partition, choose Availability or Consistency** — that is CAP, and it covers the rare case. **Else, choose Latency or Consistency** — and that is the common case, about which CAP says nothing at all. Spanner is PC/EC: it gives up availability under partition and gives up latency otherwise (the commit wait of Module 03). Dynamo is PA/EL: available under partition, and fast rather than consistent the rest of the time. **Two decisions, not one — and the second is made on every single request**, while the first applies during incidents measured in minutes per year. **PACELC is the more useful framing and is far less widely known**, which is a reasonable thing to fix in your own team's vocabulary.

### 4 Two generals

Two generals must attack simultaneously. Messengers may be captured. Can they agree on a time?

```
A sends "attack at dawn"
  -- A does not know it arrived, so A cannot commit.
B acknowledges
  -- B does not know the ACK arrived, so B cannot commit.
A acknowledges the acknowledgement
  -- A does not know THAT arrived...
```

NO FINITE PROTOCOL ACHIEVES CERTAINTY. The last message sent is always unacknowledged, and its sender cannot know it landed. Provable, for any number of messages.

WHAT YOU DO INSTEAD:

- \* at-least-once delivery + idempotence (Module 01)
- \* a third party both sides trust -- a coordinator, or in a game, THE SERVER (Module 11)
- \* accept a bounded probability of disagreement
- \* RESTRUCTURE so agreement is not required <- usually best, usually untried

**This is Module 01's timeout ambiguity, proved rather than observed.** It is the oldest result in this module and the one that bites most often in ordinary engineering — every time two services must both do something or neither, every distributed transaction, every 'did the payment go through' question. **And the fourth response is usually the right one:** a design in which one side can proceed unilaterally and the other reconciles afterwards avoids the problem rather than paying for it.

**HOW TO RECOGNISE THESE BEING MISUSED**

'**CAP means we cannot have transactions.**' No — it means not during a partition, and only for linearizability. Transactions within a partition, or serialisable transactions that are not linearizable, are unaffected.

'**FLP means consensus is impossible.**' No — it means guaranteed termination is impossible for a deterministic algorithm in a fully asynchronous model. Raft is running in production on a great many systems.

'**We're AP, so consistency isn't our problem.**' No — availability during partitions says nothing whatsoever about what happens the other 99.9% of the time, which is the PACELC else-branch and is where your users actually live.

**Ask three questions of any invocation of these results: which consistency model is meant, under what timing assumptions, and during a partition or outside one? Most invocations do not survive those questions** — and being able to ask them calmly is the practical skill this module is for.

## Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

| Resource                                                                                                                   | What it covers for this module                                                                  |
|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Gilbert & Lynch — Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services (free) | <b>The actual proof</b> , with the actual definitions. Read the definitions section at minimum. |
| Brewer — CAP Twelve Years Later: How the Rules Have Changed (free)                                                         | <b>The author correcting the folklore</b> , which is the most persuasive item in §1.            |
| Fischer, Lynch & Paterson — Impossibility of Distributed Consensus with One Faulty Process (free)                          | FLP itself. Short, and the proof idea is followable.                                            |
| Abadi — Consistency Tradeoffs in Modern Distributed Database System Design (PACELC) (free)                                 | <b>The §3 framing</b> , with real systems classified.                                           |

## Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **State CAP precisely** in your own words, with both definitions, and then state three things it does not say.
- 2 Find a vendor's marketing page invoking CAP and assess whether it uses the terms correctly.

- 3 **Partition a three-node cluster** and observe what the minority side does. Classify the system.
- 4 Reconfigure it for the other choice and observe again.
- 5 **Construct the FLP scenario informally:** delay messages so a leader election repeatedly splits. How long can you prevent progress?
- 6 Add randomised timeouts and measure how that changes.
- 7 **Classify five systems you use by PACELC.**
- 8 Implement the two generals exchange and **instrument it to show that the final message is always unacknowledged.**
- 9 Take a two-service agreement problem from your own work and **restructure it so agreement is not required.**
- 10 Write one paragraph explaining to a colleague why 'we chose AP' is not a complete answer.

## Self-check

---

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State CAP precisely, including what C and A mean.
- 2 Why is P not a choice?
- 3 Give five common misreadings of CAP.
- 4 State FLP and say what kind of impossibility it is.
- 5 How do Raft and Paxos escape it?
- 6 State PACELC and say why the else-branch matters more.
- 7 Explain the two generals problem and four responses to it.
- 8 What three questions should you ask when someone cites these results?